VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

Vansh Singh Rawat (1BF24CS330)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Vansh Singh Rawat (1BF24CS330), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026

. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

Code:

=>FCFS:

```
#include <stdio.h>

void sort(int pid[], int at[], int bt[], int n)
{
    int i, j, temp;

    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(at[j] < at[i])
            {
                temp = at[i];
                at[i] = at[j];
                at[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                temp = pid[i];
                pid[i] = pid[j];
                pid[j] = temp;
            }
        }
    }
}

int main()
{
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
 int pid[n], at[n], bt[n], ct[n], tat[n], wt[n];

    for(i = 0; i < n; i++)
        pid[i] = i + 1;

    printf("Enter arrival times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter burst times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    sort(pid, at, bt, n);

    if(at[0] > 0)
        ct[0] = at[0] + bt[0];
    else
        ct[0] = bt[0];

    for(i = 1; i < n; i++)
    {
        if(ct[i - 1] >= at[i])
            ct[i] = ct[i - 1] + bt[i];
        else
            ct[i] = at[i] + bt[i];
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    double avg_tat = 0, avg_wt = 0;

    printf("\nFCFS Scheduling\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);

        avg_tat += tat[i];
        avg_wt += wt[i];
    }

    avg_tat /= n;
    avg_wt /= n;

    printf("\nAverage Turnaround Time = %.2lf ms\n", avg_tat);
    printf("Average Waiting Time    = %.2lf ms\n", avg_wt);

    return 0;
}
```

Result:

```
Enter number of processes: 4
Enter arrival times:
0 1 2 3
Enter burst times:
5 3 8 6

FCFS Scheduling
PID AT  BT  CT  TAT WT
1   0   5   5   5   0
2   1   3   8   7   4
3   2   8   16  14  6
4   3   6   22  19  13

Average Turnaround Time = 11.25 ms
Average Waiting Time    = 5.75 ms
```

=>SJF(Non-preemptive):

```
#include <stdio.h>

struct Process
{
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
    int done;
};

int main()
{
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

 struct Process p[n];

    for(int i = 0; i < n; i++)
    {
        printf("\nEnter Arrival Time and Burst Time for P%d: ", i + 1);

        p[i].pid = i + 1;
        scanf("%d %d", &p[i].at, &p[i].bt);

        p[i].done = 0;
    }
```

```
int completed = 0;
    int time = 0;

    float totalWT = 0;
    float totalTAT = 0;

    while(completed < n)
    {
        int idx = -1;
        int minBT = 9999;

        for(int i = 0; i < n; i++)
        {
            if(p[i].at <= time && p[i].done == 0)
            {
                if(p[i].bt < minBT)
                {
                    minBT = p[i].bt;
                    idx = i;
                }
            }
        }

        if(idx == -1)
        {
            time++;
        }
 else
        {
            time += p[idx].bt;

            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            totalWT += p[idx].wt;
            totalTAT += p[idx].tat;

            p[idx].done = 1;
            completed++;
        }
    }
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for(int i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i].pid,
                p[i].at,
                p[i].bt,
                p[i].ct,
                p[i].tat,
                p[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", totalTAT / n);
    printf("\nAverage Waiting Time = %.2f\n", totalWT / n);

    return 0;
}
```

```
Enter number of processes: 4

Enter Arrival Time and Burst Time for P1: 1 2

Enter Arrival Time and Burst Time for P2: 3 2

Enter Arrival Time and Burst Time for P3: 4 4

Enter Arrival Time and Burst Time for P4: 2 3

PID AT  BT  CT  TAT WT
P1  1   2   3   2   0
P2  3   2   5   2   0
P3  4   4   12  8   4
P4  2   3   8   6   3

Average Turnaround Time = 4.50
Average Waiting Time = 1.75
```

SJF(PREEMPTIVE):

```
#include <stdio.h>

struct Process {
int pid;
   int at;     // Arrival Time
int bt;     // Burst Time     int rt;
// Remaining Time     int ct;
// Completion Time     int tat;
// Turnaround Time     int wt;
// Waiting Time
};
```

```
int main() {
int n;
printf("1BF24CS330\n");

   printf("Enter number of processes: ");
scanf("%d", &n);

   struct Process p[n];

   for(int i = 0; i < n; i++) {
      printf("\nEnter Arrival Time and Burst Time for P%d: ", i + 1);

      p[i].pid = i + 1;
      scanf("%d %d", &p[i].at, &p[i].bt);

      p[i].rt = p[i].bt;
   }

   int completed = 0, time = 0;
float totalWT = 0, totalTAT = 0;

   while(completed < n) {
int idx = -1;         int
minRT = 9999;

      // Find process with shortest remaining time
for(int i = 0; i < n; i++) {            if(p[i].at <=
time && p[i].rt > 0) {                if(p[i].rt <
minRT) {                  minRT = p[i].rt;
idx = i;
            }
         }
      }
```

```
      // If no process has arrived
if(idx == -1) {            time++;
}        else {
          // Execute process for 1 unit time
p[idx].rt--;            time++;

          // If process completed
if(p[idx].rt == 0) {
completed++;

              p[idx].ct = time;
p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;

              totalWT += p[idx].wt;
totalTAT += p[idx].tat;
          }
       }
   }

   printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

   for(int i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
p[i].at,
p[i].bt,
p[i].ct,
p[i].tat,
p[i].wt);
   }
```

```
    printf("\nAverage Turnaround Time = %.2f", totalTAT / n);
printf("\nAverage Waiting Time = %.2f\n", totalWT / n);

    return 0;
}
```

```
1BF24CS330
Enter number of processes: 4

Enter Arrival Time and Burst Time for P1: 1 2

Enter Arrival Time and Burst Time for P2: 2 4

Enter Arrival Time and Burst Time for P3: 1 3

Enter Arrival Time and Burst Time for P4: 2 5

PID AT  BT  CT  TAT WT
P1  1   2   3   2   0
P2  2   4   10  8   4
P3  1   3   6   5   2
P4  2   5   15  13  8

Average Turnaround Time = 7.00
Average Waiting Time = 3.50
```

PRIORITY(NON-PREEMPTIVE):

```
#include <stdio.h>

int main() {
```

```
    int n, i, j;

    printf("Enter number of processes: ");
scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n];
int ct[n], tat[n], wt[n];     int
completed[n];

    for(i = 0; i < n; i++)
    {
        printf("\nEnter details for Process P%d\n", i + 1);

        pid[i] = i + 1;

        printf("Arrival Time: ");
scanf("%d", &at[i]);

        printf("Burst Time: ");
scanf("%d", &bt[i]);

        printf("Priority: ");
scanf("%d", &pr[i]);

        completed[i] = 0;
    }

    int current_time = 0;
int completed_count = 0;

    while(completed_count < n)
    {
```

```
        int highest_priority = 9999;
int selected = -1;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= current_time && completed[i] == 0)
            {
                if(pr[i] < highest_priority)
                {
                    highest_priority = pr[i];
selected = i;
                }
            }
        }

        if(selected == -1)
        {
            current_time++;
        }
else
        {
            ct[selected] = current_time + bt[selected];

            tat[selected] = ct[selected] - at[selected];
wt[selected] = tat[selected] - bt[selected];

            current_time = ct[selected];

            completed[selected] = 1;
completed_count++;
        }
    }
```

```
    printf("\n----- Priority Scheduling (Non-Preemptive) -----\n");

    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    float total_tat = 0, total_wt = 0;

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                pid[i], at[i], bt[i], pr[i],
ct[i], tat[i], wt[i]);

        total_tat += tat[i];
total_wt += wt[i];
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}
```

```
1BF24CS330
Enter number of processes: 4

Enter details for Process P1
Arrival Time: 1
Burst Time: 2
Priority: 2

Enter details for Process P2
Arrival Time: 1
Burst Time: 3
Priority: 1

Enter details for Process P3
Arrival Time: 2
Burst Time: 4
Priority: 3

Enter details for Process P4
Arrival Time: 2
Burst Time: 4
Priority: 4

----- Priority Scheduling (Non-Preemptive) -----
PID AT  BT  PR  CT  TAT WT
P1  1   2   2   6   5   3
P2  1   3   1   4   3   0
P3  2   4   3   10  8   4
P4  2   4   4   14  12  8

Average Turnaround Time = 7.00
Average Waiting Time = 3.75
```

PRIORITY(PREEMPTIVE):

```
#include <stdio.h>

int main() {
    int n, i;
    printf("Enter number of processes: ");
scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n];
    int rt[n], ct[n], tat[n], wt[n];
```

```
   for(i = 0; i < n; i++)
   {
      printf("\nEnter details for Process P%d\n", i + 1);

      pid[i] = i + 1;

      printf("Arrival Time: ");
scanf("%d", &at[i]);

      printf("Burst Time: ");
scanf("%d", &bt[i]);

      printf("Priority: ");
scanf("%d", &pr[i]);
       rt[i] =
bt[i];
   }

   int current_time = 0;
int completed = 0;

   while(completed < n)
   {
      int highest_priority = 9999;
int selected = -1;

      for(i = 0; i < n; i++)
      {
         if(at[i] <= current_time && rt[i] > 0)
         {
            if(pr[i] < highest_priority)
            {
```

```
                highest_priority = pr[i];
selected = i;
            }
        }
    }

    if(selected == -1)
    {
        current_time++;
    }       else
{
rt[selected]--;

        current_time++;

        if(rt[selected] == 0)
        {
            ct[selected] = current_time;

            tat[selected] = ct[selected] - at[selected];

            wt[selected] = tat[selected] - bt[selected];

            completed++;
        }
    }
  }

  printf("\n----- Priority Scheduling (Preemptive) -----\n");
printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

  float total_tat = 0, total_wt = 0;
```

```
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                pid[i], at[i], bt[i], pr[i],
ct[i], tat[i], wt[i]);

        total_tat += tat[i];
total_wt += wt[i];
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}
```

```
Enter number of processes: 3

Enter details for Process P1
Arrival Time: 1
Burst Time: 2
Priority: 2

Enter details for Process P2
Arrival Time: 3
Burst Time: 2
Priority: 1

Enter details for Process P3
Arrival Time: 4
Burst Time: 2
Priority: 3

----- Priority Scheduling (Preemptive) -----

PID AT  BT  PR  CT  TAT WT
P1  1   2   2   3   2   0
P2  3   2   1   5   2   0
P3  4   2   3   7   3   1

Average Turnaround Time = 2.33
Average Waiting Time = 0.33
```

ROUND ROBIN:

```
#include <stdio.h>

int main() {
    int n, tq, i;
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);

    printf("Enter Time Quantum: ");
scanf("%d", &tq);

    int pid[n], at[n], bt[n], rt[n];
int ct[n], tat[n], wt[n];

    for(i = 0; i < n; i++)
    {
        printf("\nEnter details for Process P%d\n", i + 1);

        pid[i] = i + 1;

        printf("Arrival Time: ");
scanf("%d", &at[i]);

        printf("Burst Time: ");
scanf("%d", &bt[i]);
         rt[i] =
bt[i];
    }

    int completed = 0, current_time = 0;

    while(completed < n)
    {         int executed
= 0;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= current_time && rt[i] > 0)
```

```
            {
executed = 1;
                if(rt[i] >
tq)
                {
current_time += tq;
rt[i] -= tq;                }
else
                {
current_time += rt[i];

                    ct[i] = current_time;

                    tat[i] = ct[i] - at[i];

                    wt[i] = tat[i] - bt[i];
                     rt[i] =
0;

                    completed++;
                }
            }
        }

        if(executed == 0)
        {
            current_time++;
        }
    }

    printf("\n----- Round Robin Scheduling -----\n");
```

```
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    float total_tat = 0, total_wt = 0;

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                pid[i], at[i], bt[i],
ct[i], tat[i], wt[i]);

        total_tat += tat[i];
total_wt += wt[i];
    }

    printf("\nAverage Turnaround Time = %.2f",
            total_tat / n);

    printf("\nAverage Waiting Time = %.2f\n",
            total_wt / n);

    return 0;
}
```

```
1BF24CS330
Enter number of processes: 3
Enter Time Quantum: 2

Enter details for Process P1
Arrival Time: 1
Burst Time: 2

Enter details for Process P2
Arrival Time: 1
Burst Time: 4

Enter details for Process P3
Arrival Time: 2
Burst Time: 5

----- Round Robin Scheduling -----

PID AT  BT  CT  TAT WT
P1  1   2   3   2   0
P2  1   4   9   8   4
P3  2   5   12  10  5

Average Turnaround Time = 6.67
Average Waiting Time = 3.00
```

PROGRAM-2

MULTI-LEVEL QUEUE

```
#include <stdio.h>

struct Process {
int pid;
   int qn;      // Queue Number
int at;      // Arrival Time
```

```
    int bt;      // Burst Time
    int rt;      // Remaining Time
int ct;      // Completion Time
int tat;     // Turnaround Time
int wt;      // Waiting Time
};

int main() {
int n, tq;

   printf("1BF24CS330\n");
printf("Enter number of processes: ");
scanf("%d", &n);

    struct Process p[n];

    printf("Enter Time Quantum for Queue 1: ");
scanf("%d", &tq);

    // Input
    for(int i = 0; i < n; i++) {
p[i].pid = i + 1;

        printf("\nEnter Queue No, Arrival Time and Burst Time for P%d: ", i+1);
scanf("%d %d %d", &p[i].qn, &p[i].at, &p[i].bt);

        p[i].rt = p[i].bt;
    }

    int completed = 0, time = 0;
float avgTAT = 0, avgWT = 0;     //
```

```
while(completed < n) {

    int executed = 0;

    // Queue 1 - Round Robin (Higher Priority)
for(int i = 0; i < n; i++) {

        if(p[i].qn == 1 && p[i].at <= time && p[i].rt > 0) {

            executed = 1;

            if(p[i].rt > tq) {
time += tq;
p[i].rt -= tq;
            }               else
{                   time +=
p[i].rt;                    p[i].rt =
0;

                p[i].ct = time;
p[i].tat = p[i].ct - p[i].at;
p[i].wt = p[i].tat - p[i].bt;

                avgTAT += p[i].tat;
avgWT += p[i].wt;
completed++;
            }
        }
    }
```

```
      // Queue 2 - FCFS (Executed only if Queue 1 empty)
if(!executed) {            int idx = -1;            int earliest =
9999;

          for(int i = 0; i < n; i++) {
             if(p[i].qn == 2 && p[i].at <= time && p[i].rt > 0) {
if(p[i].at < earliest) {                        earliest = p[i].at;
idx = i;
                 }
             }
          }

          if(idx != -1) {
time += p[idx].rt;
p[idx].rt = 0;

             p[idx].ct = time;
p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;

             avgTAT += p[idx].tat;
avgWT += p[idx].wt;
completed++;
          }
else {
time++;
          }
       }
   }


   printf("\nPID\tQNo\tAT\tBT\tCT\tTAT\tWT\n");
```

```
    for(int i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
p[i].qn,
p[i].at,
p[i].bt,
p[i].ct,
p[i].tat,
p[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", avgTAT/n);
printf("\nAverage Waiting Time = %.2f\n", avgWT/n);

    return 0;
```

```
1BF24CS330
Enter number of processes: 4
Enter Time Quantum for Queue 1: 2

Enter Queue No, Arrival Time and Burst Time for P1: 1 4 2

Enter Queue No, Arrival Time and Burst Time for P2: 2 1 3

Enter Queue No, Arrival Time and Burst Time for P3: 1 3 4

Enter Queue No, Arrival Time and Burst Time for P4: 2 2 3

PID QNo AT  BT  CT  TAT WT
P1  1   4   2   6   2   0
P2  2   1   3   4   3   0
P3  1   3   4   10  7   3
P4  2   2   3   13  11  8

Average Turnaround Time = 5.75
Average Waiting Time = 2.75
```

PROGRAM-3

RATE MONOTONIC

```
#include <stdio.h>

struct Process {
int pid;     int
burst;     int
period;     int
remaining;
};

int main() {
   int n, time, hyper = 20;

   printf("1BF24CS330\n");
printf("Enter number of processes: ");
scanf("%d", &n);


   struct Process p[n];

   for(int i = 0; i < n; i++) {
      printf("\nEnter Burst Time and Period for P%d: ", i + 1);

      p[i].pid = i + 1;
      scanf("%d %d", &p[i].burst, &p[i].period);

      p[i].remaining = 0;
   }

   printf("\nGantt Chart:\n");

   for(time = 0; time < hyper; time++) {
```

```
        // Release process at start of period
for(int i = 0; i < n; i++) {
if(time % p[i].period == 0) {
p[i].remaining = p[i].burst;
            }
        }

        int idx = -1;         int
minPeriod = 9999;

        // Higher priority = smaller period
for(int i = 0; i < n; i++) {
            if(p[i].remaining > 0 && p[i].period < minPeriod) {
minPeriod = p[i].period;
                idx = i;
            }
        }

        if(idx != -1) {
            printf("P%d ", p[idx].pid);
p[idx].remaining--;
        }
else {
            printf("Idle ");
        }
    }

    return 0;
}
```

```
1BF24CS330
Enter number of processes: 4

Enter Burst Time and Period for P1: 1 2

Enter Burst Time and Period for P2: 2 4

Enter Burst Time and Period for P3: 2 3

Enter Burst Time and Period for P4: 1 5

Gantt Chart:
P1 P3 P1 P3 P1 P3 P1 P3 P1 P3 P1 P3 P1 P3 P1 P3 P1 P3 P1 P3
```

Earliest-deadline First :

```
#include <stdio.h>

struct Process
{    int pid;
int burst;    int
period;    int
deadline;    int
remaining;
};

int main() {    int n,
time, hyper = 20;
printf("1BF24CS330\n");
   printf("Enter number of processes: ");
scanf("%d", &n);

   struct Process p[n];

   for(int i = 0; i < n; i++) {        printf("\nEnter Burst Time
and Period for P%d: ", i + 1);

       p[i].pid = i + 1;        scanf("%d %d",
&p[i].burst, &p[i].period);

       p[i].remaining = 0;
p[i].deadline = p[i].period;
   }
```

```
    printf("\nGantt Chart:\n");

    for(time = 0; time < hyper; time++) {

        // Release tasks         for(int i = 0; i <
n; i++) {             if(time % p[i].period ==
0) {                 p[i].remaining = p[i].burst;
p[i].deadline = time + p[i].period;
            }
        }

        int idx = -1;
int earliest = 9999;

        // Select earliest deadline         for(int i = 0; i < n;
i++) {             if(p[i].remaining > 0 && p[i].deadline <
earliest) {                 earliest = p[i].deadline;                 idx
= i;
            }
        }

        if(idx != -1) {
printf("P%d ", p[idx].pid);
p[idx].remaining--;
        }         else {
printf("Idle ");
        }
    }

    return 0;
}
```

```
1BF24CS330
Enter number of processes: 3

Enter Burst Time and Period for P1: 1 2

Enter Burst Time and Period for P2: 2 4

Enter Burst Time and Period for P3: 2 6

Gantt Chart:
P1 P2 P1 P2 P1 P3 P1 P2 P1 P2 P1 P2 P1 P2 P1 P2 P1 P3 P1 P2
```

Proportional scheduling

```
#include <stdio.h>

struct Process {
int pid;     int
share;
};

int main() {
int n, total = 0;
printf("1BF24CS330\n");

   printf("Enter number of processes: ");
   scanf("%d", &n);

   struct Process p[n];

   for(int i = 0; i < n; i++) {         printf("\nEnter
CPU share for P%d: ", i + 1);

      p[i].pid = i + 1;
scanf("%d", &p[i].share);
```

```
        total += p[i].share;
    }

    printf("\nCPU Allocation:\n");

    for(int i = 0; i < n; i++) {

        float percent =
            ((float)p[i].share / total) * 100;

        printf("P%d -> %.2f%% CPU Time\n",
               p[i].pid,
percent);
    }

    return 0;
}
```

```
1BF24CS330
Enter number of processes: 3

Enter CPU share for P1: 24

Enter CPU share for P2: 33

Enter CPU share for P3: 44

CPU Allocation:
P1 -> 23.76% CPU Time
P2 -> 32.67% CPU Time
P3 -> 43.56% CPU Time
```

PROGRAM-4

PRODUCER-CONSUMER PROBLEM USING SEMAPHORES

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0; int
out = 0;

int empty = BUFFER_SIZE;
int full = 0;
```

```
void produce() {
if (empty == 0) {
        printf("\nBuffer is full! Cannot produce.\n");
return;
    }

    int item;
    printf("Enter item to produce: ");
scanf("%d", &item);

    buffer[in] = item;
    printf("Produced: %d at index %d\n", item, in);

    in = (in + 1) % BUFFER_SIZE;
empty--;     full++;
}

void consume() {
    if (full == 0) {
        printf("\nBuffer is empty! Cannot consume.\n");
return;
    }

    int item = buffer[out];
    printf("Consumed: %d from index %d\n", item, out);

    out = (out + 1) % BUFFER_SIZE;
    full--;
    empty++;
}
```

```
int main() {     int choice;
printf("1BF24CS330\n");

   printf("Buffer Size: %d\n", BUFFER_SIZE);

   while (1) {
      printf("\n1. Produce\n2. Consume\n3. Exit\n");
      printf("Current State: [Full slots: %d | Empty slots: %d]\n", full, empty);
printf("Enter choice: ");

      if (scanf("%d", &choice) != 1) break;

      switch (choice) {
case 1:                produce();
break;            case 2:
consume();                break;
case 3:
printf("Exiting...\n");
exit(0);            default:
            printf("Invalid choice!\n");
      }
   }

   return 0;
}
```

```
1BF24CS330
Buffer Size: 5

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 0 | Empty slots: 5]
Enter choice: 1
Enter item to produce: 2
Produced: 2 at index 0

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 1
Enter item to produce: 3
Produced: 3 at index 1

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 2 | Empty slots: 3]
Enter choice: 2
Consumed: 2 from index 0
```

DINNING PHILOSOPHERS PROBLEM

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#define N 5 // Number of philosophers
pthread_mutex_t forks[N]; // One mutex per
fork pthread_t philosophers[N]; void*
philosopher(void* num)
{
int id = *(int*)num; int left
= id; // left fork index
int right = (id + 1) % N; // right fork index
while (1) {
```

```
printf("Philosopher %d is thinking.\n", id);
sleep(1); // Thinking
// To avoid deadlock, philosophers with even id pick left then right, //
odd id pick right then left.
if (id % 2 == 0)
{
// Pick up left fork
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);
// Pick up right fork

pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); } else
{ // Pick up right fork
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up left fork
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);
} //
Eating
printf("Philosopher %d is eating.\n", id); sleep(2);
// Put down forks
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
} return
NULL;
}

int main() {
int i;
```

```
int ids[N]; // Initialize forks (mutexes) for
(i = 0; i < N; i++)
{ pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}
// Create philosopher threads for
(i = 0; i < N; i++)

{
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
} // Join threads (will never happen here, but good practice) for
(i = 0; i < N; i++)
{
pthread_join(philosophers[i], NULL); }
// Destroy mutexes (unreachable here)
for (i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
} return
0;
}
```

```
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
Consumed: 10 from buffer[0]
Produced: 15 at buffer[0]
Consumed: 11 from buffer[1]
Produced: 16 at buffer[1]
Consumed: 12 from buffer[2]
Produced: 17 at buffer[2]
Consumed: 13 from buffer[3]
Produced: 18 at buffer[3]
```

PROGRAM-5
BANKER'S ALGORITHM

```
#include<stdio.h>

int main() {
   int allocation[10][10], max[10][10], need[10][10];
int available[10], finish[10], safeSequence[10];
int i, j, k, p, r, count = 0;

   printf("1BF24CS330\n");

   printf("Enter number of processes: ");
scanf("%d", &p);

   printf("Enter number of resources: ");
scanf("%d", &r);

   printf("Enter Allocation Matrix:\n");
for(i = 0; i < p; i++)
   {       for(j = 0; j < r;
j++)
      {
         scanf("%d", &allocation[i][j]);
      }
   }

   printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < p; i++)
   {       for(j = 0; j < r;
j++)       {
         scanf("%d", &max[i][j]);
      }
   }
```

```
   printf("Enter Available Resources:\n");
for(i = 0; i < r; i++)
   {
      scanf("%d", &available[i]);
   }

   // Calculate Need Matrix
for(i = 0; i < p; i++)
   {       for(j = 0; j < r;
j++)
      {
         need[i][j] = max[i][j] - allocation[i][j];
      }
   }

   // Initialize finish array
for(i = 0; i < p; i++)
   {       finish[i]
= 0;
   }

   while(count < p)
   {       int found
= 0;

      for(i = 0; i < p; i++)
      {
         if(finish[i] == 0)
         {              for(j = 0;
j < r; j++)
            {
```

```
                if(need[i][j] > available[j])
                {
break;
                }
            }
if(j == r)
{
                for(k = 0; k < r; k++)
                {
                   available[k] += allocation[i][k];
                }

                safeSequence[count] = i;
count++;                    finish[i] = 1;
found = 1;
            }
         }
      }

      if(found == 0)
      {
         printf("\nSystem is not in a safe state.\n");
return 0;
      }
   }

   printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

   for(i = 0; i < p; i++)
   {
      printf("P%d", safeSequence[i]);
```

```
        if(i != p - 1)
        {
            printf(" -> ");
        }
    }

    return 0;
}
```

```
1BF24CS330
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 2 0
0 3 4
1 2 0
3 0 0
0 0 8
Enter Maximum Demand Matrix:
8 4 2
2 5 6
3 5 0
2 0 4
3 6 8
Enter Available Resources:
2 4 6

System is not in a safe state.
```

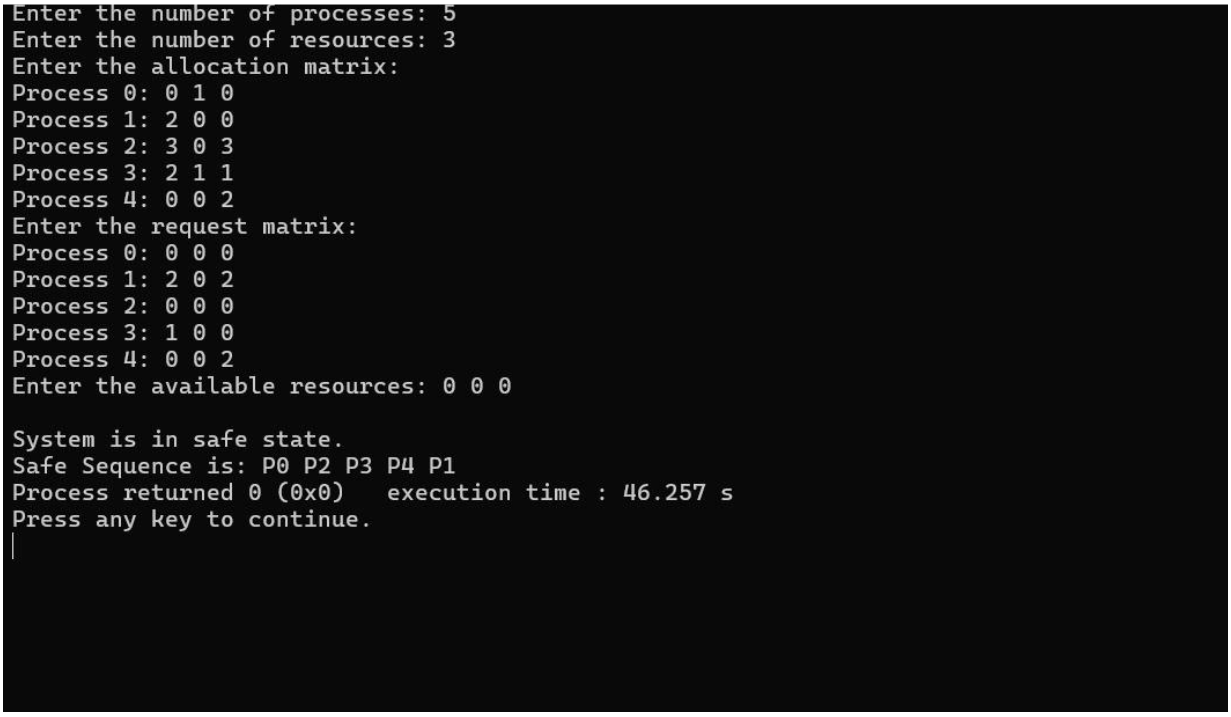
DEADLOCK DETECTION:

```
#include<stdio.h>

int main() {
    int allocation[10][10], request[10][10];
int available[10];     int finish[10],
safeSequence[10];     int i, j, k, p, r, count
= 0;
```

```
    printf("USN-1BF24CS330\n");

   printf("Enter the number of processes: ");
scanf("%d", &p);

   printf("Enter the number of resources: ");
scanf("%d", &r);

   // Allocation Matrix     printf("Enter
the allocation matrix:\n");     for(i = 0; i <
p; i++)
   {        printf("Process %d:
", i);

       for(j = 0; j < r; j++)
       {
          scanf("%d", &allocation[i][j]);
       }
   }

   // Request Matrix     printf("Enter
the request matrix:\n");     for(i = 0; i <
p; i++)
   {
       printf("Process %d: ", i);

       for(j = 0; j < r; j++)
       {
          scanf("%d", &request[i][j]);
       }
   }
```

```
   // Available Resources     printf("Enter
the available resources: ");     for(i = 0; i <
r; i++)
   {
      scanf("%d", &available[i]);
   }

   // Initialize finish array
for(i = 0; i < p; i++)
   {        finish[i]
= 0;
   }

   while(count < p)
   {        int found
= 0;

       for(i = 0; i < p; i++)
       {
if(finish[i] == 0)
           {               for(j = 0;
j < r; j++)
               {
                  if(request[i][j] > available[j])
{                       break;
                  }
               }               if(j == r)
{                  for(k = 0; k < r;
k++)
                  {
                     available[k] += allocation[i][k];
```

```
                }

                safeSequence[count++] = i;
finish[i] = 1;                  found = 1;
            }
          }
       }

       if(found == 0)
       {
break;
       }
   }

   if(count == p)
   {
      printf("\nSystem is in safe state.\n");
printf("Safe Sequence is: ");

       for(i = 0; i < p; i++)
       {
          printf("P%d ", safeSequence[i]);
       }
}
else
   {
      printf("\nDeadlock detected in the system.\n");
   }

   return 0;
}
```



PROGRAM-6

FIRST FIT, BEST FIT, WORST FIT

```
#include <stdio.h>


void firstFit(int blockSize[], int blocks, int processSize[], int processes) {
int allocation[processes];



   for (int i = 0; i < processes; i++)
allocation[i] = -1;

   int used[blocks];     for (int i
= 0; i < blocks; i++)
used[i] = 0;
```

```
   for (int i = 0; i < processes; i++) {
for (int j = 0; j < blocks; j++) {

          // block must be unused
          if (!used[j] && blockSize[j] >= processSize[i]) {
allocation[i] = j;
              used[j] = 1; // mark block as used
break;
          }
       }
   }

   printf("\n--- First Fit ---\n");
   printf("Process No.\tProcess Size\tBlock No.\n");

   for (int i = 0; i < processes; i++) {
printf("%d\t\t%d\t\t", i + 1, processSize[i]);

       if (allocation[i] != -1)
printf("%d\n", allocation[i] + 1);
else
          printf("Not Allocated\n");
   }
}

void bestFit(int blockSize[], int blocks, int processSize[], int processes) {
int allocation[processes];

   for (int i = 0; i < processes; i++)
allocation[i] = -1;
```

```
   int used[blocks];     for (int i
= 0; i < blocks; i++)
used[i] = 0;

   for (int i = 0; i < processes; i++) {
int bestIdx = -1;

       for (int j = 0; j < blocks; j++) {

           if (!used[j] && blockSize[j] >= processSize[i]) {

               if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
bestIdx = j;
               }
           }
       }

       if (bestIdx != -1) {
allocation[i] = bestIdx;
used[bestIdx] = 1;
       }
   }

   printf("\n--- Best Fit ---\n");
   printf("Process No.\tProcess Size\tBlock No.\n");

   for (int i = 0; i < processes; i++) {
printf("%d\t\t%d\t\t", i + 1, processSize[i]);

       if (allocation[i] != -1)
printf("%d\n", allocation[i] + 1);
else
```

```
            printf("Not Allocated\n");
        }
    }

    void worstFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[processes];

        for (int i = 0; i < processes; i++)
    allocation[i] = -1;

        int used[blocks];     for (int i
    = 0; i < blocks; i++)
    used[i] = 0;

        for (int i = 0; i < processes; i++) {
    int worstIdx = -1;

            for (int j = 0; j < blocks; j++) {

                if (!used[j] && blockSize[j] >= processSize[i]) {

                    if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
    worstIdx = j;
                    }
                }
            }

            if (worstIdx != -1) {
    allocation[i] = worstIdx;
    used[worstIdx] = 1;
            }
        }
```

```
    printf("\n--- Worst Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < processes; i++) {
printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
printf("%d\n", allocation[i] + 1);
else
            printf("Not Allocated\n");
    }
}

int main() {     int blocks,
processes;
printf("1BF24CS330\n");

    printf("Enter number of memory blocks: ");
scanf("%d", &blocks);

    int blockSize[blocks];

    printf("Enter sizes of memory blocks:\n");
for (int i = 0; i < blocks; i++) {
scanf("%d", &blockSize[i]);
    }

    printf("Enter number of processes: ");
scanf("%d", &processes);

    int processSize[processes];
```

```
    printf("Enter sizes of processes:\n");
for (int i = 0; i < processes; i++) {
scanf("%d", &processSize[i]);
    }

    firstFit(blockSize, blocks, processSize, processes);
bestFit(blockSize, blocks, processSize, processes);
worstFit(blockSize, blocks, processSize, processes);

    return 0;
}
```

```
1BF24CS330
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

--- First Fit ---
Process No. Process Size    Block No.
1       212     2
2       417     5
3       112     3
4       426     Not Allocated

--- Best Fit ---
Process No. Process Size    Block No.
1       212     4
2       417     2
3       112     3
4       426     5

--- Worst Fit ---
Process No. Process Size    Block No.
1       212     5
2       417     2
3       112     4
4       426     Not Allocated
```

PROGRAM-7

Write a C program to simulate page replacement algorithms. a) FIFO b) LRU
c) Optimal

```
#include <stdio.h>

#define MAX 50

// Function to check if page exists in frames int
isPresent(int frames[], int num_frames, int page) {
```

```
    for (int i = 0; i < num_frames; i++)
    {       if (frames[i] ==
page)           return 1;    }
return 0;
}

// Function to print frames void printFrames(int
frames[], int num_frames)
{    printf("[");    for (int i = 0; i <
num_frames; i++)
    {       if (frames[i] != -1)
printf("%d", frames[i]);
else            printf(" ");

      if (i != num_frames - 1)
printf(" ");    }
printf("]\n");
}

// ---------------- FIFO ---------------- void
FIFO(int pages[], int n, int num_frames)
{    int
frames[MAX];    int
page_faults = 0;
int next = 0;

    for (int i = 0; i < num_frames; i++)
frames[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n\n");

    for (int i = 0; i < n; i++)
```

```
    {
        int page = pages[i];

        if (!isPresent(frames, num_frames, page))
        {
            frames[next] = page;            next
= (next + 1) % num_frames;
page_faults++;
        }

        printf("Page %d -> ", page);
printFrames(frames, num_frames);
    }

    printf("\nTotal Page Faults (FIFO): %d\n", page_faults);
}

// ---------------- LRU ---------------- void
LRU(int pages[], int n, int num_frames)
{    int frames[MAX];
int last_used[MAX];
int page_faults = 0;

    for (int i = 0; i < num_frames; i++)
    {        frames[i] = -
1;        last_used[i] =
-1;
    }

    printf("\n--- LRU Page Replacement ---\n\n");

    for (int i = 0; i < n; i++)
```

```
    {        int page =
pages[i];        int found
= 0;

       // Check if page exists         for (int
j = 0; j < num_frames; j++)
       {            if (frames[j]
== page)
           {               found
= 1;
last_used[j] = i;
break;
           }
       }

       // Page Fault
if (!found)        {
int pos = -1;

           // Empty frame available
for (int j = 0; j < num_frames; j++)
           {               if
(frames[j] == -1)               {
pos = j;                   break;
              }
           }

           // Replace least recently used
if (pos == -1)
           {
              int min = last_used[0];
pos = 0;
```

```
            for (int j = 1; j < num_frames; j++)
            {
               if (last_used[j] < min)
               {
                  min = last_used[j];
pos = j;
               }
            }
         }

         frames[pos] = page;
last_used[pos] = i;
page_faults++;
      }

      printf("Page %d -> ", page);
printFrames(frames, num_frames);
   }

   printf("\nTotal Page Faults (LRU): %d\n", page_faults); }

// ---------------- Optimal ---------------- void
Optimal(int pages[], int n, int num_frames)
{    int
frames[MAX];    int
page_faults = 0;

   for (int i = 0; i < num_frames; i++)
frames[i] = -1;

   printf("\n--- Optimal Page Replacement ---\n\n");
```

```
   for (int i = 0; i < n; i++)
   {        int page =
pages[i];

       if (!isPresent(frames, num_frames, page))
       {            int
pos = -1;

          // Empty frame            for (int j =
0; j < num_frames; j++)
          {               if
(frames[j] == -1)
             {
pos = j;
break;
             }
          }

          // Find optimal replacement
          if (pos == -1)
{               int farthest = -
1;

             for (int j = 0; j < num_frames; j++)
             {                  int k;
for (k = i + 1; k < n; k++)
                {
                   if (frames[j] == pages[k])
break;
                }
```

```
                // Page never used again
if (k == n)                 {
pos = j;                    break;
                }

                // Farthest use
if (k > farthest)
{                   farthest =
k;                   pos = j;
                }
            }
        }

        frames[pos] = page;
page_faults++;
    }

    printf("Page %d -> ", page);
printFrames(frames, num_frames);
  }

  printf("\nTotal Page Faults (Optimal): %d\n", page_faults);
}



int main() {    int n, num_frames;
int pages[MAX];
printf("1BF24CS330\n");

  printf("Enter number of pages: ");
scanf("%d", &n);
```

```
    printf("Enter page reference string:\n");
for (int i = 0; i < n; i++)          scanf("%d",
&pages[i]);

    printf("Enter number of frames: ");
scanf("%d", &num_frames);

    FIFO(pages, n, num_frames);
    LRU(pages, n, num_frames);
    Optimal(pages, n, num_frames);

    return 0;
}
```

```
1BF24CS330
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---

Page 7 -> [7    ]
Page 0 -> [7 0  ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]

Total Page Faults (FIFO): 10

--- LRU Page Replacement ---

Page 7 -> [7    ]
Page 0 -> [7 0  ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [4 0 3]
Page 2 -> [4 0 2]
Page 3 -> [4 3 2]
Page 0 -> [0 3 2]
Page 3 -> [0 3 2]

Total Page Faults (LRU): 9
--- Optimal Page Replacement ---

Page 7 -> [7    ]
Page 0 -> [7 0  ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]

Total Page Faults (Optimal): 7
```